



PYTHON

STARTTHECODE



[DATE]
[COMPANY NAME]
[Company address]

Comprehensive Python Course Outline

Week 1-2: Introduction to Python

Module 1: Getting Started with Python

- Introduction to Python and its history
- Installing Python and setting up the development environment
- Using Python IDEs (PyCharm, VS Code, Jupyter Notebook)

Module 2: Python Basics

- Syntax and semantics
- Variables and data types (int, float, string, boolean)
- Basic operations (arithmetic, comparison, logical)
- Input and output functions (`input()`, `print()`)

Week 3-4: Control Structures and Functions

Module 3: Control Structures

- Conditional statements (`if`, `elif`, `else`) , Loops (`for`, `while`)
- Loop control statements (`break`, `continue`, `pass`)

Module 4: Functions

- Defining and calling functions
- Function arguments and return values
- Scope and lifetime of variables
- Recursion

Week 5-6: Data Structures

Module 5: Lists and Tuples

- Creating and accessing lists
- List methods and operations
- List comprehensions
- Tuples and their uses

Module 6: Dictionaries and Sets

- Creating and accessing dictionaries
- Dictionary methods and operations
- Sets and their uses

- Set operations

Week 7-8: Modules, Packages, and File Handling

Module 7: Modules and Packages PROJECT Weather using pip library

- Importing modules and using standard libraries
- Creating and using custom modules
- Installing and using third-party packages (pip)

Module 8: File Handling

- Reading from and writing to files
- Error handling and exceptions

Week 9-10: Object-Oriented Programming (OOP)

Module 9: OOP Basics

- Classes and objects
- Methods and attributes
- `__init__` method (constructor)
- Inheritance and polymorphism
- Encapsulation and abstraction

Module 10: Advanced OOP Concepts

- Magic methods and operator overloading
- Class and static methods
- Exception classes

Week 11-12: Practical Applications and Projects

Module 13: Real-World Applications

- GUI development with Tkinter

Python Programming Language

Python is a widely used programming language that offers several unique features and advantages compared to languages like **Java** and **C++**. Our Python tutorial thoroughly explains Python basics and advanced concepts, starting with [installation](#), [conditional statements](#), [loops](#), [built-in datastructures](#), [Object-Oriented Programming](#), [Generators](#), [Exception Handling](#), [Python RegEx](#), and many other concepts. This tutorial is designed for beginners and working professionals.

In the late 1980s, **Guido van Rossum** dreamed of developing Python. The first version of **Python 0.9.0 was released in 1991**. Since its release, Python started gaining popularity. According to reports, Python is now the most popular programming language among developers because of its high demands in the tech realm.

Features of Python:

- **Easy to use and Read** - Python's syntax is clear and easy to read, making it an ideal language for both beginners and experienced programmers. This simplicity can lead to faster development and reduce the chances of errors.
- **Dynamically Typed** - The data types of variables are determined during run-time. We do not need to specify the data type of a variable during writing codes.
- **High-level** - High-level language means human readable code.
- **Compiled and Interpreted** - Python code first gets compiled into bytecode, and then interpreted line by line. When we download the Python in our system form [org](#) we download the default implement of Python known as CPython. CPython is considered to be Compiled and Interpreted both.

Java vs. Python

Python is an excellent choice for rapid development and scripting tasks. Whereas Java emphasizes a strong type system and object-oriented programming.

Here are some basic programs that illustrates key differences between them.

Printing 'Hello World'

Python Code:

1. `print("Hello World")`

In Python, it is one line of code. It requires simple syntax to print 'Hello World'

Getting Started with Python

Python is a versatile, high-level programming language that is widely supported across all major operating systems.

To execute Python code, you need to have a Python interpreter installed on your system. However, if you want to start immediately, you can use our free [online Python editor](#) that enables you to run Python code directly in your browser—no installation required.

For those who prefer to install Python on your computer, this guide will walk you through the installation process on Windows, macOS, and Linux (Ubuntu).

[Windows](#)

[Linux](#)

[MacOS](#)

Install Python on Windows

To install Python on your Windows, just follow these steps:

1. Install VS Code
2. Download Python Installer File

3. Run the Installer
4. Install Python
5. Verify your installation

Here is a detailed explanation of each of the steps:

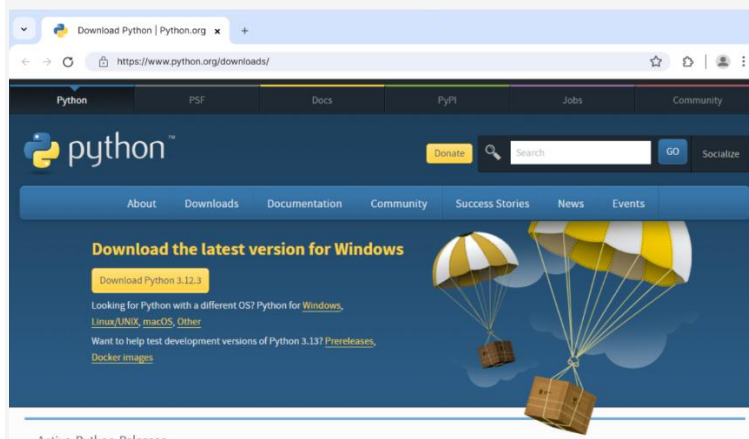
Step 1: Install VS Code

Go to the VS Code [Official website](#) and download the Windows installer. Once the download is complete, run the installer and follow the installation process. Click **Finish** to complete the installation process

Step 2: Download the Python Installer File

Go to the official [Python website](#) and download the latest version (Python 3.12.2 at the time of writing this tutorial) for Windows.

The website automatically detects your operating system and gives you the right installer.



Active Python Releases
Python Download Page

Step 3: Run the Installer

Now, go to your **download** folder and run the installer you just downloaded. Depending on your security settings, you might be prompted to allow access.

Simply allow it and proceed.

Step 4: Install Python

Once you have run the installer, you will come across this screen.



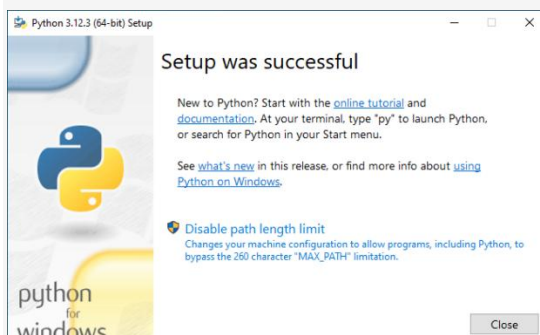
Install Python on Windows

On the screen, you will see two options: **Install Now** and **Customize Installation**. We suggest you skip all customization steps and simply click **Install Now**.

- Check on **Add python.exe to PATH** as it ensures Python is added to our system's PATH variable. (Recommended)
- Click **Install Now**, as it will include all the necessary files needed later.

This makes it easier to run a Python Program from the command prompt (cmd) directly without specifying the full path of the Python executable.

After using this option, Python will be successfully installed in your device.

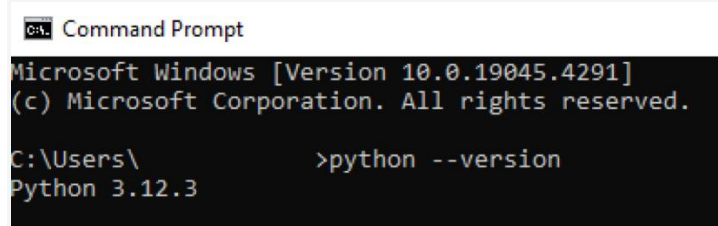


Python Default Installation

Step 4: Verify your installation

After the installation is complete, you can verify whether Python is installed by using the following command in the command prompt.

```
python --version
```



```
Microsoft Windows [Version 10.0.19045.4291]
(c) Microsoft Corporation. All rights reserved.

C:\Users\>python --version
Python 3.12.3
```

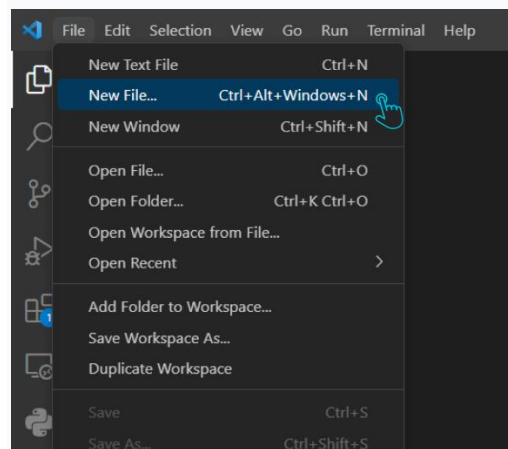
Python Installation Verify

Note: The version number might differ from the one above, depending on your installed version.

Now, you are all set to run Python programs on your device.

Run Your First Python Program

First open VS Code, click on the File in the top menu and then select New File.



Create a New File in VS Code

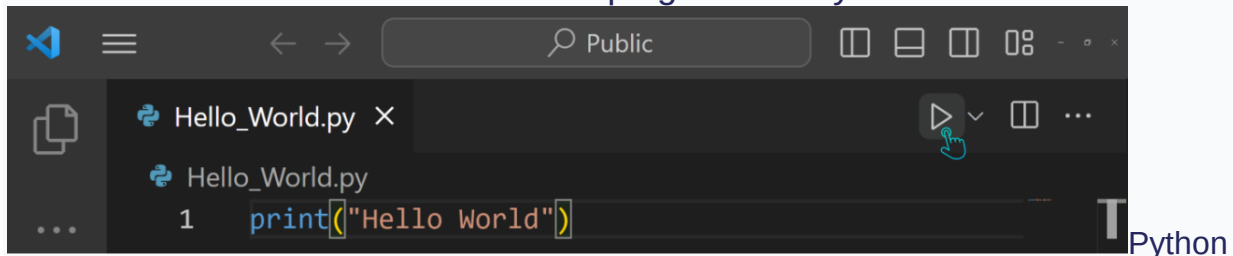
Then, save this file with a .py extension by clicking on **File** again, then **Save As**, and type your filename ending in .py. (Here, we are saving it as Hello_World.py)

Before you start coding, make sure the Python extension is installed in VS Code. Open VS Code and click on Extensions on the left sidebar. Then, search for the **Python** extension by Microsoft and click on install.

Now, write the following code into your file:

```
print("Hello World")
```

Then click on the **run** button on the top right side of your screen.



Run Program

You should see Hello World printed to the command prompt.

Your First Python Program

In the previous tutorial, you learned how to [install Python](#) on your computer.

Now, let's write a simple Python program.

The following program displays `Hello, World!` on the screen.

```
print("Hello, World!")
```

Run Code

Output

```
Hello World!
```

Note: A Hello World program is used to introduce a new programming language. It is the first program that every beginner writes.

It's okay if you don't understand how the program works. We will learn all about them in upcoming tutorials. For now, just write the exact program and run it.

Working of the Program

Congratulations on writing your first Python program. Now, let's see how the above program works.

```
print("Hello, World!")
```

Hello World Code

In Python, anything inside `print()` is displayed on the screen.

There are two things to note about `print()`:

- Everything we want to display on the screen is included inside the parentheses `()`.
- The text we want to print is placed within double quotes `" "`.

We can also use single quotes to print text on the screen. For example,

```
print('Hello World!')
```

Run Code

is same as

```
print("Hello World!")
```

Run Code

To be consistent, we will be using double quotes throughout the tutorials.

Next, we will be learning about [Python comments](#).

Python Comments

In the previous tutorial, you learned to write your first [Python program](#). Now, let's learn about Python comments.

Important!: We are introducing comments early in this tutorial series because we will be using them to explain the code in upcoming tutorials.

Comments are hints that we add to our code to make it easier to understand. Python comments start with `#`. For example,

```
# print a number  
print(25)  
Run Code
```

Here, `# print a number` is a comment.

Comments are completely ignored and not executed by code editors.

Important: The purpose of this tutorial is to help you understand comments, so you can ignore other concepts used in the program. We will learn about them in later tutorials.

Single-line Comment

We use the **hash (#)** symbol to write a single-line comment. For example,

```
# declare a variable  
name = "John"  
  
# print name  
print(name)    # John  
Run Code
```

In the above example, we have used three single-line comments:

- `# declare a variable`
- `# print name`

- `# John`

A **single-line comment** starts with `#` and extends up to the end of the line.

We can also use single-line comments alongside the code:

```
print(name)    # John
```

Note: Remember the keyboard shortcut to apply comments. In most text editors, it's **Ctrl + /** if you are on Windows & **Cmd + /** if you are on a Mac.

Multiline Comments

Unlike languages such as C++ and Java, Python doesn't have a dedicated method to write multi-line comments.

However, we can achieve the same effect by using the hash (`#`) symbol at the beginning of each line.

Let's look at an example.

```
# This is an example of a multiline comment
# created using multiple single-line comments
# The code prints the text Hello World
print("Hello, World!")
Run Code
```

We can also use multiline strings as comments like:

```
'''This is an example
of multiline comment'''
print("Hello, World!")
Run Code
```

Output

```
Hello World
```

Note: Remember you will learn about these programming concepts in upcoming tutorials. For now, you can just focus on the usage of comments.

Prevent Executing Code Using Comments

Comments are valuable when debugging code.

If we encounter an error while running a program, instead of removing code segments, we can comment them out to prevent execution. For example,

```
number1 = 10
number2 = 15

sum = number1 + number2

print("The sum is:", sum)
print("The product is:", product)
Run Code
```

Here, the code throws an error because we have not defined a `product` variable. Instead of removing the line causing an error, we can comment it.

For example,

```
number1 = 10
number2 = 15

sum = number1 + number2

print("The sum is:", sum)
# print('The product is:', product)
Run Code
```

Output

```
The sum is 25
```

Here, the code runs without any errors.

We have resolved the error using a comment. Now if you need to calculate the `product` in the near future, you can uncomment it.

Note: This approach comes in handy while working with large files. Instead of deleting any line, we can use comments and identify which one is causing an error.

Why Use Comments?

We should use comments:

- For future references, as comments make our code readable.
- For debugging.
- For code collaboration, as comments help peer developers to understand each other's code.

Note: Comments are not and should not be used as a substitute to explain poorly written code. Always try to write clean, understandable code, and then use comments as an addition.

In most cases, always use comments to explain 'why' rather than 'how' and you are good to go.

Next, we will start learning fundamental concepts of Python programming.

Python Variables and Literals

In the previous tutorial you learned about [Python comments](#). Now, let's learn about variables and literals in Python.

Python Variables

In programming, a variable is a container (storage area) to hold data. For example,

```
number = 10
```

Here, `number` is a variable storing the value `10`.

Assigning values to Variables in Python

As we can see from the above example, we use the assignment operator `=` to assign a value to a variable.

```
# assign value to site_name variable
site_name = 'programiz.pro'

print(site_name)

# Output: programiz.pro
Run Code
```

Output

```
apple.com
```

In the above example, we assigned the value `programiz.pro` to the `site_name` variable. Then, we printed out the value assigned to `site_name`

Note: Python is a [type-inferred](#) language, so you don't have to explicitly define the variable type. It automatically knows that `programiz.pro` is a string and declares the `site_name` variable as a string.

Changing the Value of a Variable in Python

```
site_name = 'programiz.pro'
```

```
print(site_name)

# assigning a new value to site_name
site_name = 'apple.com'

print(site_name)
Run Code
```

Output

```
programiz.pro
apple.com
```

Here, the value of `site_name` is changed from `'programiz.pro'` to `'apple.com'`.

Example: Assigning multiple values to multiple variables

```
a, b, c = 5, 3.2, 'Hello'

print (a) # prints 5
print (b) # prints 3.2
print (c) # prints Hello
Run Code
```

If we want to assign the same value to multiple variables at once, we can do this as:

```
site1 = site2 = 'programiz.com'

print (x) # prints programiz.com
print (y) # prints programiz.com
Run Code
```

Here, we have assigned the same string value `'programiz.com'` to both the variables `site1` and `site2`.

Rules for Naming Python Variables

Python Literals

Literals are representations of fixed values in a program. They can be numbers, characters, or strings, etc. For example, `'Hello, World!'`, `12`, `23.0`, `'C'`, etc.

Literals are often used to assign values to variables or constants. For example,

```
site_name = 'programiz.com'
```

In the above expression, `site_name` is a variable, and `'programiz.com'` is a literal. There are different types of literals in Python. Let's discuss some of the commonly used types in detail.

Python Numeric Literals

Numeric Literals are immutable (unchangeable). Numeric literals can belong to **3** different numerical types: `Integer`, `Float`, and `Complex`.

1. Integer Literals

Integer literals are numbers without decimal parts. It also consists of negative numbers. For example, `5`, `-11`, `0`, `12`, etc.

2. Floating-Point Literals

Floating-point literals are numbers that contain decimal parts.

Just like integers, floating-point numbers can also be both positive and negative. For example, `2.5`, `6.76`, `0.0`, `-9.45`, etc.

3. Complex Literals

Complex literals are numbers that represent complex numbers.

Here, numerals are in the form `a + bj`, where `a` is real and `b` is imaginary. For example, `6+9j`, `2+3j`.

Python String Literals

In Python, texts wrapped inside quotation marks are called **string literals**..

```
"This is a string."
```

We can also use single quotes to create strings.

```
'This is also a string.'
```

Python Type Conversion

In programming, type conversion is the process of converting data of one type to another. For example: converting `int` data to `str`.

There are two types of type conversion in Python.

- Implicit Conversion - automatic type conversion
- Explicit Conversion - manual type conversion

Python Implicit Type Conversion

In certain situations, Python automatically converts one [data type](#) to another. This is known as implicit type conversion.

Example 1: Converting integer to float

Let's see an example where Python promotes the conversion of the lower data type (integer) to the higher data type (float) to avoid data loss.

```
integer_number = 123
```

```
float_number = 1.23

new_number = integer_number + float_number

# display new value and resulting data type
print("Value:", new_number)
print("Data Type:", type(new_number))
```

[Run Code](#)

Output

```
Value: 124.23
Data Type: <class 'float'>
```

In the above example, we have created two variables: `integer_number` and `float_number` of `int` and `float` type respectively. Then we added these two variables and stored the result in `new_number`. As we can see `new_number` has value **124.23** and is of the `float` data type. It is because Python always converts smaller data types to larger data types to avoid the loss of data.

Note:

- We get `TypeError`, if we try to add `str` and `int`. For example, `'12' + 23`. Python is not able to use Implicit Conversion in such conditions.
- Python has a solution for these types of situations which is known as Explicit Conversion.

Explicit Type Conversion

In Explicit Type Conversion, users convert the data type of an object to required data type.

We use the built-in functions like [int\(\)](#), [float\(\)](#), [str\(\)](#), etc to perform explicit type conversion.

This type of conversion is also called typecasting because the user casts (changes) the data type of the objects.

Example 2: Addition of string and integer Using Explicit Conversion

```
num_string = '12'
num_integer = 23

print("Data type of num_string before Type Casting:",type(num_string))

# explicit type conversion
num_string = int(num_string)

print("Data type of num_string after Type Casting:",type(num_string))

num_sum = num_integer + num_string

print("Sum:",num_sum)
print("Data type of num_sum:",type(num_sum))
Run Code
```

Output

```
Data type of num_string before Type Casting: <class 'str'>
Data type of num_string after Type Casting: <class 'int'>
Sum: 35
Data type of num_sum: <class 'int'>
```

In the above example, we have created two variables: `num_string` and `num_integer` with `str` and `int` type values respectively. Notice the code,

```
num_string = int(num_string)
```

Here, we have used `int()` to perform explicit type conversion of `num_string` to integer type.

After converting `num_string` to an integer value, Python is able to add these two variables.

Finally, we got the `num_sum` value i.e **35** and data type to be `int`.

Key Points to Remember

1. Type Conversion is the conversion of an object from one data type to another data type.
2. Implicit Type Conversion is automatically performed by the Python interpreter.
3. Python avoids the loss of data in Implicit Type Conversion.
4. Explicit Type Conversion is also called Type Casting, the data types of objects are converted using predefined functions by the user.
5. In Type Casting, loss of data may occur as we enforce the object to a specific data type.

Python Basic Input and Output

Python Output

In Python, we can simply use the [print\(\)](#) function to print output. For example,

```
print('Python is powerful')
```

Output: Python is powerful

[Run Code](#)

Here, the `print()` function displays the string enclosed inside the single quotation.

Syntax of print()

In the above code, the `print()` function is taking a single parameter.

However, the actual syntax of the print function accepts **5** parameters

```
print(object= separator= end= file= flush=)
```

Here,

- **object** - value(s) to be printed
- **sep** (optional) - allows us to separate multiple **objects** inside `print()`.
- **end** (optional) - allows us to add add specific values like new line `"\n"`, tab `"\t"`
- **file** (optional) - where the values are printed. It's default value is `sys.stdout` (screen)
- **flush** (optional) - boolean specifying if the output is flushed or buffered.

Default: `False`

Example 1: Python Print Statement

```
print('Good Morning!')
print('It is rainy today')
```

[Run Code](#)

Output

```
Good Morning!
It is rainy today
```

In the above example, the `print()` statement only includes the **object** to be printed. Here, the value for **end** is not used. Hence, it takes the default value `'\n'`.

So we get the output in two different lines.

Example 2: Python print() with end Parameter

```
# print with end whitespace
print('Good Morning!', end= ' ')
```



```
print('It is rainy today')
```

[Run Code](#)

Output

```
Good Morning! It is rainy today
```

Notice that we have included the `end= ' '` after the end of the first `print()` statement.

Hence, we get the output in a single line separated by space.

Example 3: Python print() with sep parameter

```
print('New Year', 2023, 'See you soon!', sep= '. ')
```

Output

```
New Year. 2023. See you soon!
```

In the above example, the `print()` statement includes multiple **items** separated by a comma.

Notice that we have used the optional parameter `sep= ". "` inside the `print()` statement.

Hence, the output includes items separated by `.` not comma.

Example: Print Python Variables and Literals

We can also use the `print()` function to print [Python variables](#). For example,

```
number = -10.6

name = "Programiz"

# print literals
print(5)

# print variables
print(number)
print(name)
Run Code
```

Output

```
5
-10.6
Programiz
```

Example: Print Concatenated Strings

We can also join two [strings](#) together inside the `print()` statement. For example,

```
print('Programiz is ' + 'awesome.')
Run Code
```

Output

```
Programiz is awesome.
```

Here,

- the `+` operator joins two strings `'Programiz is '` and `'awesome.'`
- the `print()` function prints the joined string

Output formatting

Sometimes we would like to format our output to make it look attractive. This can be done by using the `str.format()` method. For example,

```
x = 5
y = 10

print('The value of x is {} and y is {}'.format(x,y))
Run Code
```

Here, the curly braces `{}` are used as placeholders. We can specify the order in which they are printed by using numbers (tuple index).

To learn more about formatting the output, visit [Python String format\(\)](#).

Python Input

While programming, we might want to take the input from the user. In Python, we can use the [input\(\)](#) function.

Syntax of input()

```
input(prompt)
```

Here, `prompt` is the string we wish to display on the screen. It is optional.

Example: Python User Input

```
# using input() to take user input
num = input('Enter a number: ')

print('You Entered:', num)

print('Data type of num:', type(num))
```

[Run Code](#)

Output

```
Enter a number: 10
You Entered: 10
Data type of num: <class 'str'>
```

In the above example, we have used the `input()` function to take input from the user and stored the user input in the `num` variable.

It is important to note that the entered value **10** is a string, not a number.

So, `type(num)` returns `<class 'str'>`.

To convert user input into a number we can use [int\(\)](#) or [float\(\)](#) functions as:

```
num = int(input('Enter a number: '))
```

Here, the [data type](#) of the user input is converted from string to integer .

Python Operators

Operators are special symbols that perform operations on [variables](#) and values. For example,

```
print(5 + 6)    # 11
```

[Run Code](#)

Here, `+` is an operator that adds two numbers: **5** and **6**.

Types of Python Operators

Here's a list of different types of Python operators that we will learn in this tutorial.

1. Arithmetic Operators
2. Assignment Operators
3. Comparison Operators
4. Logical Operators
5. Bitwise Operators
6. Special Operators

1. Python Arithmetic Operators

Arithmetic operators are used to perform mathematical operations like addition, subtraction, multiplication, etc. For example,

```
sub = 10 - 5 # 5
```

Here, `-` is an arithmetic operator that subtracts two values or variables.

Operator	Operation	Example
<code>+</code>	Addition	<code>5 + 2 = 7</code>

-

Subtraction

4 - 2 = 2

*

Multiplication

2 * 3 = 6

/

Division

4 / 2 = 2

//

Floor Division

10 // 3 = 3

%

Modulo

5 % 2 = 1

**

Power

4 ** 2 = 16

Example 1: Arithmetic Operators in Python

```
a = 7
b = 2

# addition
print ('Sum: ', a + b)

# subtraction
print ('Subtraction: ', a - b)

# multiplication
print ('Multiplication: ', a * b)

# division
print ('Division: ', a / b)

# floor division
print ('Floor Division: ', a // b)

# modulo
print ('Modulo: ', a % b)

# a to the power b
print ('Power: ', a ** b)
Run Code
```

Output

```
Sum: 9
Subtraction: 5
Multiplication: 14
Division: 3.5
Floor Division: 3
Modulo: 1
Power: 49
```

In the above example, we have used multiple arithmetic operators,

- `+` to add `a` and `b`
- `-` to subtract `b` from `a`
- `*` to multiply `a` and `b`
- `/` to divide `a` by `b`
- `//` to floor divide `a` by `b`
- `%` to get the remainder
- `**` to get `a` to the power `b`
-

2. Python Assignment Operators

Assignment operators are used to assign values to variables. For example,

```
# assign 5 to x
x = 5
```

Here, `=` is an assignment operator that assigns **5** to `x`.

Here's a list of different assignment operators available in Python.

Operator	Name	Example
<code>=</code>	Assignment Operator	<code>a = 7</code>

`+=`

Addition Assignment

`a += 1 # a = a + 1``-=`

Subtraction Assignment

`a -= 3 # a = a - 3``*=`

Multiplication Assignment

`a *= 4 # a = a * 4``/=`

Division Assignment

`a /= 3 # a = a / 3``%=`

Remainder Assignment

`a %= 10 # a = a % 10``**=`

Exponent Assignment

`a **= 10 # a = a ** 10`

Example 2: Assignment Operators

```
# assign 10 to a
a = 10

# assign 5 to b
b = 5

# assign the sum of a and b to a
a += b      # a = a + b

print(a)

# Output: 15
Run Code
```

Here, we have used the `+=` operator to assign the sum of `a` and `b` to `a`. Similarly, we can use any other assignment operators as per our needs.

3. Python Comparison Operators

Comparison operators compare two values/variables and return a boolean result: `True` or `False`. For example,

```
a = 5
b = 2

print (a > b)    # True
Run Code
```

Here, the `>` comparison operator is used to compare whether `a` is greater than `b` or not.

Operator	Meaning	Example
<code>==</code>	Is Equal To	<code>3 == 5</code> gives us <code>False</code>
<code>!=</code>	Not Equal To	<code>3 != 5</code> gives us <code>True</code>
<code>></code>	Greater Than	<code>3 > 5</code> gives us <code>False</code>
<code><</code>	Less Than	<code>3 < 5</code> gives us <code>True</code>
<code>>=</code>	Greater Than or Equal To	<code>3 >= 5</code> give us <code>False</code>
<code><=</code>	Less Than or Equal To	<code>3 <= 5</code> gives us <code>True</code>

Example 3: Comparison Operators

```
a = 5
b = 2

# equal to operator
print('a == b =', a == b)

# not equal to operator
print('a != b =', a != b)

# greater than operator
print('a > b =', a > b)

# less than operator
print('a < b =', a < b)
```

```
# greater than or equal to operator
print('a >= b =', a >= b)
```

```
# less than or equal to operator
print('a <= b =', a <= b)
```

[Run Code](#)

Output

```
a == b = False
a != b = True
a > b = True
a < b = False
a >= b = True
a <= b = False
```

Note: Comparison operators are used in decision-making and [loops](#). We'll discuss more of the comparison operator and decision-making in later tutorials.

4. Python Logical Operators

Logical operators are used to check whether an expression is `True` or `False`. They are used in decision-making. For example,

```
a = 5
b = 6
```

```
print((a > 2) and (b >= 6))    # True
```

[Run Code](#)

Here, `and` is the logical operator **AND**. Since both `a > 2` and `b >= 6` are `True`, the result is `True`.

Operator	Example	Meaning
<code>and</code>	<code>a and b</code>	Logical AND: <code>True</code> only if both the operands are <code>True</code>

or

a or b

Logical OR:

True if at least one of the operands is True

not

not a

Logical NOT:

True if the operand is False and vice-versa.

Example 4: Logical Operators

```
# logical AND
print(True and True)    # True
print(True and False)   # False

# logical OR
print(True or False)    # True

# logical NOT
print(not True)         # False
Run Code
```

Note: Here is the [truth table](#) for these logical operators.

5. Python Bitwise operators

Bitwise operators act on operands as if they were strings of binary digits. They operate bit by bit, hence the name.

For example, 2 is 10 in binary, and 7 is 111.

In the table below: Let x = 10 (0000 1010 in binary) and y = 4 (0000 0100 in binary)

Operator	Meaning	Example
&	Bitwise AND	x & y = 0 (0000 0000)
	Bitwise OR	x y = 14 (0000 1110)
~	Bitwise NOT	~x = -11 (1111 0101)

<code>^</code>	Bitwise XOR	$x \wedge y = 14$ (0000 1110)
<code>>></code>	Bitwise right shift	$x \gg 2 = 2$ (0000 0010)
<code><<</code>	Bitwise left shift	$x \ll 2 = 4$ (0010 1000)

6. Python Special operators

Python language offers some special types of operators like the **identity** operator and the **membership** operator. They are described below with examples.

Identity operators

In Python, `is` and `is not` are used to check if two values are located at the same memory location.

It's important to note that having two variables with equal values doesn't necessarily mean they are identical.

Operator	Meaning	Example
<code>is</code>	True if the operands are identical (refer to the same object)	<code>x is True</code>
<code>is not</code>	True if the operands are not identical (do not refer to the same object)	<code>x is not True</code>

Example 4: Identity operators in Python

```
x1 = 5
y1 = 5
x2 = 'Hello'
y2 = 'Hello'
x3 = [1,2,3]
y3 = [1,2,3]

print(x1 is not y1) # prints False
```

```
print(x2 is y2) # prints True

print(x3 is y3) # prints False
Run Code
```

Here, we see that `x1` and `y1` are integers of the same values, so they are equal as well as identical. The same is the case with `x2` and `y2` (strings). But `x3` and `y3` are lists. They are equal but not identical. It is because the interpreter locates them separately in memory, although they are equal.

Membership operators

In Python, `in` and `not in` are the membership operators. They are used to test whether a value or variable is found in a sequence ([string](#), [list](#), [tuple](#), [set](#) and [dictionary](#)).

In a dictionary, we can only test for the presence of a key, not the value.

Operator	Meaning	Example
<code>in</code>	True if value/variable is found in the sequence	<code>5 in x</code>
<code>not in</code>	True if value/variable is not found in the sequence	<code>5 not in x</code>

Example 5: Membership operators in Python

```
message = 'Hello world'
dict1 = {1:'a', 2:'b'}

# check if 'H' is present in message string
print('H' in message) # prints True

# check if 'hello' is present in message string
print('hello' not in message) # prints True

# check if '1' key is present in dict1
print(1 in dict1) # prints True

# check if 'a' key is present in dict1
print('a' in dict1) # prints False
```

[Run Code](#)

Output

```
True
True
True
False
```

Here, 'H' is in `message`, but 'hello' is not present in `message` (remember, Python is case-sensitive).

Similarly, `1` is key, and 'a' is the value in dictionary `dict1`. Hence, 'a' in `y` returns

```
if condition:
    # body of if statement

else:
    # body of else statement
```

Python if...elif...else Statement

The `if...else` statement is used to execute a block of code among two alternatives. However, if we need to make a choice between more than two alternatives, we use the `if...elif...else` statement.

Syntax

```
if condition1:
    # code block 1

elif condition2:
    # code block 2
```

```
else:
    # code block 3
```

Here,

- `if condition1` - This checks if `condition1` is `True`. If it is, the program executes **code block 1**.
- `elif condition2` - If `condition1` is not `True`, the program checks `condition2`. If `condition2` is `True`, it executes **code block 2**.
- `else` - If neither `condition1` nor `condition2` is `True`, the program defaults to executing **code block 3**.

Let's look at an example.

Example: Python if...elif...else Statement

```
number = 0

if number > 0:
    print('Positive number')

elif number < 0:
    print('Negative number')

else:
    print('Zero')

print('This statement is always executed')
```

Output

```
Zero
This statement is always executed
```

Python Nested if Statements

It is possible to include an `if` statement inside another `if` statement. For example,

```
number = 5

# outer if statement
if number >= 0:
    # inner if statement
    if number == 0:
        print('Number is 0')

    # inner else statement
    else:
        print('Number is positive')

# outer else statement
else:
    print('Number is negative')
```

Output

```
Number is positive
```

Python for Loop

In Python, a `for` loop is used to iterate over sequences such as [lists](#), [strings](#), [tuples](#), etc.

```
languages = ['Swift', 'Python', 'Go']

# access elements of the list one by one
for i in languages:
    print(i)
```

Output

```
Swift
Python
Go
```

In the above example, we have created a list called `languages`. As the list has 3 elements, the loop iterates 3 times.

The value of `i` is

- `Swift` in the first iteration.
- `Python` in the second iteration.
- `Go` in the third iteration.

for loop Syntax

```
for val in sequence:  
    # statement(s)
```

Here, `val` accesses each item of the `sequence` on each iteration. The loop continues until we reach the last item in the sequence.

Flowchart of Python for Loop

Example: Loop Through a String

```
language = 'Python'  
  
# iterate over each character in language  
for x in language:  
    print(x)
```

P
y
t
h
o
n

Here, we have printed each character of the string `language` using a `for` loop.

for Loop with Python range()

In Python, the [range\(\)](#) function returns a sequence of numbers. For example,

```
values = range(4)
```

Here, `range(4)` returns a sequence of **0**, **1**, **2**, and **3**.

Since the `range()` function returns a sequence of numbers, we can iterate over it using a `for` loop. For example,

```
# iterate from i = 0 to i = 3
for i in range(4):
    print(i)
```

0

1

2

3

More on Python for Loop

A `for` loop can have an optional `else` clause. This `else` clause executes after the iteration completes.

```
digits = [0, 1, 5]

for i in digits:
    print(i)
else:
    print("No items left.")
```

Output

0

1

5

No items left.

Here, the `for` loop prints all the items of the `digits` list. When the loop finishes, it executes the `else` block and prints `No items left`.

```
languages = ['Swift', 'Python', 'Go']

# looping to repeat an action without using the list elements
for language in languages:
```

```
print('Hi')
```

[Run Code](#)

Output

```
Hi  
Hi  
Hi
```

Here, we used the list `languages` to run the loop three times. However, we didn't use any of the elements of the list.

In such cases, it is clearer to use the `_` (underscore) as the loop variable. The `_` indicates that a loop variable is a placeholder and its value is intentionally being ignored.

For example,

```
languages = ['Swift', 'Python', 'Go']  
  
# using _ for placeholder variable  
for _ in languages:  
    print('Hi')
```

Here, the loop still runs three times because there are three elements in the `languages` list.

Using `_` indicates that the loop is there for repetition and not for accessing the elements.

NESTED FOR LOOP

A `for` loop can also have another `for` loop inside it. For each cycle of the outer loop, the inner loop completes its entire sequence of iterations. For example,

```
# outer loop  
for i in range(2):  
    # inner loop  
    for j in range(2):  
        print(f"i = {i}, j = {j}")
```

[Run Code](#)

Output

```
i = 0, j = 0  
i = 0, j = 1  
i = 1, j = 0
```



```
i = 1, j = 1
```

----- PRACTICE

```
# python program to illustrate If statement
```

```
i = 10
```

```
if (i > 15):  
    print("10 is less than 15")  
print("I am Not in if")  
I am Not in if
```

```
i = 20  
if (i < 15):  
    print("i is smaller than 15")  
    print("i'm in if Block")  
else:  
    print("i is greater than 15")  
    print("i'm in else Block")  
print("i'm not in if and not in else Block")
```

i is greater than 15

i'm in else Block

i'm not in if and not in else Block

```
# python program to illustrate nested If statement
```

```
i = 10
```

```
if (i == 10):  
    # First if statement  
    if (i < 15):  
        print("i is smaller than 15")  
    # Nested - if statement  
    # Will only be executed if statement above  
    # it is true  
    if (i < 12):  
        print("i is smaller than 12 too")  
    else:  
        print("i is greater than 15")
```

i is smaller than 15
i is smaller than 12 too

```
i = 20
if (i == 10):
    print("i is 10")
elif (i == 15):
    print("i is 15")
elif (i == 20):
    print("i is 20")
else:
    print("i is not present")
```

Python break and continue

In programming, the `break` and `continue` statements are used to alter the flow of loops:

- `break` exits the loop entirely
- `continue` skips the current iteration and proceeds to the next one

Python break Statement

The `break` statement terminates the loop immediately when it's encountered.

Syntax

Working of Python break Statement

Example: break Statement with for Loop

We can use the `break` statement with the `for` loop to terminate the loop when a certain condition is met. For example,

```
for i in range(5):
    if i == 3:
        break
    print(i)
```

Output

```
0
1
2
```

In the above example,

```
if i == 3:
    break
```

terminates the loop when `i` is equal to `3`. Hence, the output doesn't include values after `2`.

Python continue Statement

The `continue` statement skips the current iteration of the loop and the control flow of the program goes to the next iteration.

Syntax

```
continue
```

Working of continue Statement in Python

Example: continue Statement with for Loop

We can use the `continue` statement with the `for` loop to skip the current iteration of the loop and jump to the next iteration. For example,

```
for i in range(5):
    if i == 3:
        continue
    print(i)
```

[Run Code](#)

Output

```
0
1
2
```

In the above example,

```
if i == 3:  
    continue
```

skips the current iteration when `i` is equal to `3`, and continues the next iteration. Hence, the output has all the values except `3`.

Python pass Statement

In Python programming, the `pass` statement is a null statement which can be used as a placeholder for future code.

Suppose we have a [loop](#) or a [function](#) that is not implemented yet, but we want to implement it in the future. In such cases, we can use the `pass` statement.

The syntax of the `pass` statement is:

```
pass
```

Using pass With Conditional Statement

```
n = 10  
  
# use pass inside if statement  
if n > 10:  
    pass  
  
print('Hello')  
Run Code
```

Here, notice that we have used the `pass` statement inside the [if statement](#).

Suppose we didn't use `pass` or just put a comment as:

```
n = 10

if n > 10:
    # write code later

print('Hello')
Run Code
```

Here, we will get an error message: `IndentationError: expected an indented block`

Note: The difference between a [comment](#) and a `pass` statement in Python is that while the interpreter ignores a comment entirely, `pass` is not ignored.

Use of pass Statement inside Function or Class

We can do the same thing in an empty function or [class](#) as well. For example,

```
def function(args):
    pass

class Example:
    pass
```

```
# Getting the number of multiplication table from user
tableNumber = int(input())

print("Printing Multiplication Table of", tableNumber, "in Reverse
Order")

# Using For loop to print Multiplication table from number 10 to 1 of
user input number
for num in range(10, 0, -1):
    print(num, "*", tableNumber, " =", (num * tableNumber))
```

```
Enter the Number of Multiplication Table to Print:
5
Printing Multiplication Table of 5 in Reverse Order:
10 * 5  = 50
9 * 5   = 45
```

```
8 * 5 = 40
7 * 5 = 35
6 * 5 = 30
5 * 5 = 25
4 * 5 = 20
3 * 5 = 15
2 * 5 = 10
1 * 5 = 5
```

Python while Loop

In Python, we use the `while` loop to repeat a block of code until a certain condition is met. For example,

```
number = 1

while number <= 3:
    print(number)
    number = number + 1
```

[Run Code](#)

Output

```
1
2
3
```

In the above example, we have used a `while` loop to print the numbers from **1** to **3**. The loop runs as long as the condition `number <= 3` is satisfied.

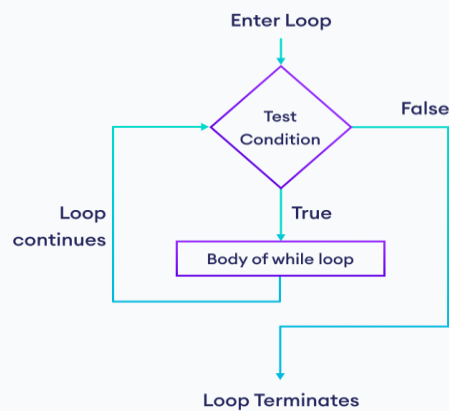
while Loop Syntax

```
while condition:
    # body of while loop
```

Here,

1. The `while` loop evaluates the condition.
2. If the condition is true, **body of while loop** is executed. The condition is evaluated again.
3. This process continues until the condition is `False`.
4. Once the condition evaluates to `False`, the loop terminates.

Flowchart of Python while Loop



Flowchart of Python while Loop

Example: Python while Loop

```
# Calculate the sum of numbers until user enters 0
number = int(input('Enter a number: '))

total = 0

# iterate until the user enters 0
while number != 0:
    total += number
    number = int(input('Enter a number: '))

print('The sum is', total)
```

[Run Code](#)

Output

```
Enter a number: 3
Enter a number: 2
Enter a number: 1
Enter a number: -4
Enter a number: 0
The sum is 2
```

Here is how the above program works:

1. It asks the user to enter a number.
2. If the user enters a number other than **0**, it adds the number to the `total` and asks the user to enter a number again.
3. If the user enters **0**, the loop terminates and the program displays the total.

Infinite while Loop

If the condition of a `while` loop is always `True`, the loop runs for infinite times, forming an **infinite while loop**. For example,

```
age = 32

# the test condition is always True
while age > 18:
    print('You can vote')
Run Code
```

Output

```
You can vote
You can vote
You can vote
.
.
.
```

The above program is equivalent to:

```
age = 32
```



```
# the test condition is always True
while True:
    print('You can vote')
```

[Run Code](#)

More on Python while Loop

Python `while` loop with `break` statement

Python `while` loop with an `else` clause

Python for loop vs while loop

The [for loop](#) is usually used in the sequence when the number of iterations is known. For example,

```
# loop is iterated 4 times
for i in range(4):
    print(i)
```

[Run Code](#)

Output

```
0
1
2
3
```

The `while` loop is usually used when the number of iterations is unknown. For example,

```
while True:
    user_input = input("Enter password: ")

    # terminate the loop when user enters exit
    if user_input == 'exit':
        print(f'Status: Entry Rejected')
        break

    print(f'Status: Entry Allowed')
```

[Run Code](#)

Output

```
Enter password: Python is Fun
```

```
Status: Entry Allowed
Enter password: exit
Status: Entry Rejected
```

Python break and continue

In programming, the `break` and `continue` statements are used to alter the flow of loops:

- `break` exits the loop entirely
- `continue` skips the current iteration and proceeds to the next one

Python break Statement

The `break` statement terminates the loop immediately when it's encountered.

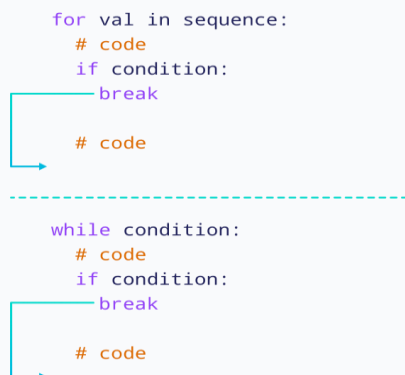
Syntax

```
break
```

Working of Python break Statement

```
for val in sequence:
    # code
    if condition:
        break
    # code
```

```
while condition:
    # code
    if condition:
        break
    # code
```



Working of break Statement in Python

The above image shows the working of break statements in [for](#) and [while](#) loops.

Note: The `break` statement is usually used inside decision-making statements such as `if...else`.

Example: break Statement with for Loop

We can use the `break` statement with the `for` loop to terminate the loop when a certain condition is met. For example,

```
for i in range(5):  
    if i == 3:  
        break  
    print(i)
```

[Run Code](#)

Output

```
0  
1  
2
```

In the above example,

```
if i == 3:  
    break
```

terminates the loop when `i` is equal to **3**. Hence, the output doesn't include values after **2**.

Note: We can also terminate the `while` loop using a `break` statement.

break Statement with while Loop

Python continue Statement

The `continue` statement skips the current iteration of the loop and the control flow of the program goes to the next iteration.

Syntax

`continue`

Working of continue Statement in Python

```
for val in sequence:  
    # code  
    if condition:  
        continue
```

```
    # code
```

```
while condition:  
    # code  
    if condition:  
        continue
```

```
    # code
```

Example: continue Statement with for Loop

We can use the `continue` statement with the `for` loop to skip the current iteration of the loop and jump to the next iteration. For example,

```
for i in range(5):  
    if i == 3:  
        continue  
    print(i)
```

[Run Code](#)

Output

```
0  
1  
2  
4
```

In the above example,

```
if i == 3:  
    continue
```

skips the current iteration when `i` is equal to `3`, and continues the next iteration. Hence, the output has all the values except `3`.

Note: We can also use the `continue` statement with a `while` loop.

Output

```
0
1
2
4
```

In the above example,

```
if i == 3:
    continue
```

skips the current iteration when `i` is equal to `3`, and continues the next iteration. Hence, the output has all the values except `3`.

Python pass Statement

In Python programming, the `pass` statement is a null statement which can be used as a placeholder for future code.

Suppose we have a [loop](#) or a [function](#) that is not implemented yet, but we want to implement it in the future. In such cases, we can use the `pass` statement.

The syntax of the `pass` statement is:

```
pass
```

Using pass With Conditional Statement

```
n = 10
```

```
# use pass inside if statement
if n > 10:
    pass

print('Hello')
Run Code
```

Here, notice that we have used the `pass` statement inside the [if statement](#) .

However, nothing happens when the pass is executed. It results in no operation (NOP).

Suppose we didn't use pass or just put a comment as:

```
n = 10

if n > 10:
    # write code later

print('Hello')
Run Code
```

Here, we will get an error message: `IndentationError: expected an indented block`

Note: The difference between a [comment](#) and a `pass` statement in Python is that while the interpreter ignores a comment entirely, `pass` is not ignored.

Use of pass Statement inside Function or Class

We can do the same thing in an empty function or [class](#) as well. For example,

```
def function(args):
    pass

class Example:
    pass
```

Python break and continue

In programming, the `break` and `continue` statements are used to alter the flow of loops:

- `break` exits the loop entirely
- `continue` skips the current iteration and proceeds to the next one

Python break Statement

The `break` statement terminates the loop immediately when it's encountered.

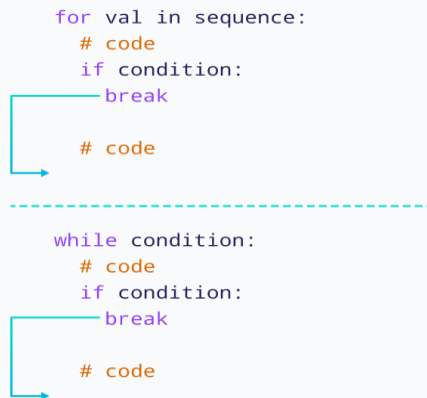
Syntax

```
break
```

Working of Python break Statement

```
for val in sequence:
    # code
    if condition:
        break
    # code
```

```
while condition:
    # code
    if condition:
        break
    # code
```



Working of break Statement in Python

The above image shows the working of break statements in [for](#) and [while](#) loops.

Note: The `break` statement is usually used inside decision-making statements such as [if...else](#).

Example: break Statement with for Loop

We can use the `break` statement with the `for` loop to terminate the loop when a certain condition is met. For example,

```
for i in range(5):
    if i == 3:
        break
```

```
print(i)
```

[Run Code](#)

Output

```
0
1
2
```

In the above example,

```
if i == 3:
    break
```

terminates the loop when `i` is equal to **3**. Hence, the output doesn't include values after **2**.

Note: We can also terminate the `while` loop using a `break` statement.

break Statement with while Loop

Python continue Statement

The `continue` statement skips the current iteration of the loop and the control flow of the program goes to the next iteration.

Syntax

```
continue
```

Working of continue Statement in Python


```
for val in sequence:
    # code
    if condition:
        continue
```

```
# code
```

```
while condition:
    # code
    if condition:
        continue
```

```
# code
```

Example: continue Statement with for Loop

We can use the `continue` statement with the `for` loop to skip the current iteration of the loop and jump to the next iteration. For example,

```
for i in range(5):
    if i == 3:
        continue
    print(i)
```

[Run Code](#)

Output

```
0
1
2
4
```

In the above example,

```
if i == 3:
    continue
```

skips the current iteration when `i` is equal to **3**, and continues the next iteration. Hence, the output has all the values except **3**.

Python pass Statement

In Python programming, the `pass` statement is a null statement which can be used as a placeholder for future code.

Suppose we have a [loop](#) or a [function](#) that is not implemented yet, but we want to implement it in the future. In such cases, we can use the `pass` statement.

The syntax of the `pass` statement is:

```
pass
```

Using pass With Conditional Statement

```
n = 10

# use pass inside if statement
if n > 10:
    pass

print('Hello')
```

Python Lists

```
mylist = ["apple", "banana", "cherry"]
```

List

Lists are used to store multiple items in a single variable.

Lists are one of 4 built-in data types in Python used to store collections of data, the other 3 are [Tuple](#), [Set](#), and [Dictionary](#), all with different qualities and usage.

Lists are created using square brackets:

Example [Get your own Python Server](#)

Create a List:

```
thislist = ["apple", "banana", "cherry"]  
print(thislist)
```

List Items

List items are ordered, changeable, and allow duplicate values.

List items are indexed, the first item has index `[0]`, the second item has index `[1]` etc.

Ordered

When we say that lists are ordered, it means that the items have a defined order, and that order will not change.

If you add new items to a list, the new items will be placed at the end of the list.

Note: There are some [list methods](#) that will change the order, but in general: the order of the items will not change.

Changeable

The list is changeable, meaning that we can change, add, and remove items in a list after it has been created.

Allow Duplicates

Since lists are indexed, lists can have items with the same value:

Example

Lists allow duplicate values:

```
thislist = ["apple", "banana", "cherry", "apple", "cherry"]  
print(thislist)
```

List Length

To determine how many items a list has, use the `len()` function:

Example

Print the number of items in the list:

```
thislist = ["apple", "banana", "cherry"]  
print(len(thislist))
```

List Items - Data Types

List items can be of any data type:

Example

String, int and boolean data types:

```
list1 = ["apple", "banana", "cherry"]  
list2 = [1, 5, 7, 9, 3]  
list3 = [True, False, False]
```

A list can contain different data types:

Example

A list with strings, integers and boolean values:

```
list1 = ["abc", 34, True, 40, "male"]
```

type()

From Python's perspective, lists are defined as objects with the data type 'list':

```
<class 'list'>
```

Example

What is the data type of a list?

```
mylist = ["apple", "banana", "cherry"]  
print(type(mylist))
```

The list() Constructor

It is also possible to use the `list()` constructor when creating a new list.

Example

Using the `list()` constructor to make a List:

```
thislist = list(("apple", "banana", "cherry")) # note the double round-  
brackets  
print(thislist)
```

Python Collections (Arrays)

There are four collection data types in the Python programming language:

- **List** is a collection which is ordered and changeable. Allows duplicate members.
- **Tuple** is a collection which is ordered and unchangeable. Allows duplicate members.
- **Set** is a collection which is unordered, unchangeable*, and unindexed. No duplicate members.
- **Dictionary** is a collection which is ordered** and changeable. No duplicate members.

```
thislist = ["apple", "banana", "cherry", "apple", "cherry"]
```

```
print(thislist)
```

```
['apple', 'banana', 'cherry', 'apple', 'cherry']
```

```
list1 = ["abc", 34, True, 40, "male"]
```

```
print(list1)
```

```
['abc', 34, True, 40, 'male']
```

```
thislist = list(("apple", "banana", "cherry"))
```

```
print(thislist)
```

```
['apple', 'banana', 'cherry']
```

Access Items

List items are indexed and you can access them by referring to the index number:

Example

Print the second item of the list:

```
thislist = ["apple", "banana", "cherry"]  
print(thislist[1])
```

banana

```
thislist = ["apple", "banana", "cherry"]  
print(thislist[1])
```

Python List

In Python, lists allow us to store a sequence of items in a single variable.

Create a Python List

We create a list by placing elements inside square brackets `[]`, separated by commas. For example,

```
# a list of three elements  
ages = [19, 26, 29]  
print(ages)
```

```
# Output: [19, 26, 29]  
Run Code
```

Here, the `ages` list has three items.

[More on List Creation](#)

[Using list\(\) to Create Lists](#)

List Characteristics

Lists are:

- **Ordered** - They maintain the order of elements.
- **Mutable** - Items can be changed after creation.
- **Allow duplicates** - They can contain duplicate values.

Access List Elements

Each element in a list is associated with a number, known as a **index**.

The index always starts from **0**. The first element of a list is at index **0**, the second element is at index **1**, and so on.

Index of List Elements

Access Elements Using Index

We use index numbers to access list elements. For example,

```
languages = ['Python', 'Swift', 'C++']
```

```
# access the first element
```

```
print(languages[0]) # Python
```

```
# access the third element
```

```
print(languages[2]) # C++
```

[Run Code](#)

Access List Elements

More on Accessing List Elements

Negative Indexing in Python

	['Python',	'Swift',	'C++']
Index →	0	1	2
Negative Index →	-3	-2	-1

Slicing of a List in Python

Note: If the specified index does not exist in a list, Python throws the `IndexError` exception.

Add Elements to a Python List

We use the [append\(\)](#) method to add an element to the end of a Python list.

For example,

```
fruits = ['apple', 'banana', 'orange']  
print('Original List:', fruits)
```

```
# using append method  
fruits.append('cherry')
```

```
print('Updated List:', fruits)  
Run Code
```

Output

```
Original List: ['apple', 'banana', 'orange']  
Updated List: ['apple', 'banana', 'orange', 'cherry']
```

Add Elements at the Specified Index

Add Elements to a List From Other Iterables

Change List Items

We can change the items of a list by assigning new values using the `=` operator. For example,

```
colors = ['Red', 'Black', 'Green']  
print('Original List:', colors)
```

```
# changing the third item to 'Blue'  
colors[2] = 'Blue'
```

```
print('Updated List:', colors)  
Run Code
```


Output

```
Original List: ['Red', 'Black', 'Green']  
Updated List: ['Red', 'Black', 'Blue']
```

Here, we have replaced the element at index 2: 'Green' with 'Blue'.

Remove an Item From a List

We can remove an item from a list using the [remove\(\)](#) method. For example,

```
numbers = [2,4,7,9]
```

```
# remove 4 from the list  
numbers.remove(4)
```

```
print(numbers)
```

```
# Output: [2, 7, 9]
```

[Run Code](#)

Remove One or More Elements of a List

Python List Length

We can use the built-in [len\(\)](#) function to find the number of elements in a list.

For example,

```
cars = ['BMW', 'Mercedes', 'Tesla']
```

```
print('Total Elements: ', len(cars))
```

```
# Output: Total Elements: 3
```

[Run Code](#)

Iterating Through a List

We can use a [for loop](#) to iterate over the elements of a list. For example,

```
fruits = ['apple', 'banana', 'orange']
```

```
# iterate through the list
```

```
for fruit in fruits:
    print(fruit)
Run Code
```

Output

```
apple
banana
orange
```

Python List Methods

Python has many useful [list methods](#) that make it really easy to work with lists.

Method	Description
append()	Adds an item to the end of the list
extend()	Adds items of lists and other iterables to the end of the list
insert()	Inserts an item at the specified index
remove()	Removes item present at the given index
pop()	Returns and removes item present at the given index
clear()	Removes all items from the list
index()	Returns the index of the first matched item
count()	Returns the count of the specified item in the list
sort()	Sorts the list in ascending/descending order

[reverse\(\)](#)

Reverses the item of the list

[copy\(\)](#)

Returns the shallow copy of the list

Python Tuple

A tuple is a collection similar to a [Python list](#). The primary difference is that we cannot modify a tuple once it is created.

Create a Python Tuple

We create a tuple by placing items inside parentheses `()`. For example,

```
numbers = (1, 2, -5)
```

```
print(numbers)
```

```
# Output: (1, 2, -5)
```

[Run Code](#)

More on Tuple Creation

Create a Tuple Using tuple() Constructor

Different Types of Python Tuples

Tuple Characteristics

Tuples are:

- **Ordered** - They maintain the order of elements.
- **Immutable** - They cannot be changed after creation.
- **Allow duplicates** - They can contain duplicate values.

Access Tuple Items

Each item in a tuple is associated with a number, known as a **index**.

The index always starts from **0**, meaning the first item of a tuple is at index **0**, the second item is at index **1**, and so on.

Tuple	→	('Python',	'Swift',	'C++')
Index	→	0	1	2

Index of Tuple Item

Access Items Using Index

We use index numbers to access tuple items. For example,

```
languages = ('Python', 'Swift', 'C++')
```

```
# access the first item
```

```
print(languages[0]) # Python
```

```
# access the third item
```

```
print(languages[2]) # C++
```

[Run Code](#)

Tuple	→	('Python',	'Swift',	'C++')
Access Tuple	→	languages[0]	languages[1]	languages[2]

Access Tuple Items

Tuple Cannot be Modified

Python tuples are immutable (unchangeable). We cannot add, change, or delete items of a tuple.

If we try to modify a tuple, we will get an error. For example,

```
cars = ('BMW', 'Tesla', 'Ford', 'Toyota')
```

```
# trying to modify a tuple
cars[0] = 'Nissan'    # error
```

```
print(cars)
```

[Run Code](#)

Python Tuple Length

We use the [len\(\)](#) function to find the number of items present in a tuple. For example,

```
cars = ('BMW', 'Tesla', 'Ford', 'Toyota')
print('Total Items:', len(cars))
```

```
# Output: Total Items: 4
```

[Run Code](#)

Iterate Through a Tuple

We use the [for loop](#) to iterate over the items of a tuple. For example,

```
fruits = ('apple', 'banana', 'orange')
```

```
# iterate through the tuple
```

```
for fruit in fruits:
    print(fruit)
```

[Run Code](#)

Output

```
apple
banana
orange
```

Python Sets

A set is a collection of unique data, meaning that elements within a set cannot be duplicated.

For instance, if we need to store information about student IDs, a set is suitable since student IDs cannot have duplicates.

112	114	116	118	115
-----	-----	-----	-----	-----

Set of Student ID

Python Set Elements

Create a Set in Python

In Python, we create sets by placing all the elements inside curly braces {}, separated by commas.

A set can have any number of items and they may be of different types (integer, float, [tuple](#), [string](#), etc.). But a set cannot have mutable elements like [lists](#), sets or [dictionaries](#) as its elements.

Let's see an example,

```
# create a set of integer type
student_id = {112, 114, 116, 118, 115}
print('Student ID:', student_id)

# create a set of string type
vowel_letters = {'a', 'e', 'i', 'o', 'u'}
print('Vowel Letters:', vowel_letters)

# create a set of mixed data types
mixed_set = {'Hello', 101, -2, 'Bye'}
print('Set of mixed data types:', mixed_set)
```

[Run Code](#)

Output

```
Student ID: {112, 114, 115, 116, 118}
Vowel Letters: {'u', 'a', 'e', 'i', 'o'}
Set of mixed data types: {'Hello', 'Bye', 101, -2}
```

In the above example, we have created different types of sets by placing all the elements inside the curly braces `{}`.

Note: When you run this code, you might get output in a different order. This is because the set has no particular order.

Create an Empty Set in Python

Creating an empty set is a bit tricky. Empty curly braces `{}` will make an empty dictionary in Python.

To make a set without any elements, we use the `set()` function without any argument. For example,

```
# create an empty set
empty_set = set()

# create an empty dictionary
empty_dictionary = { }

# check data type of empty_set
print('Data type of empty_set:', type(empty_set))

# check data type of dictionary_set
print('Data type of empty_dictionary:', type(empty_dictionary))
```

[Run Code](#)

Output

```
Data type of empty_set: <class 'set'>
Data type of empty_dictionary: <class 'dict'>
```

Here,

- `empty_set` - an empty set created using `set()`
- `empty_dictionary` - an empty dictionary created using `{}`

Finally, we have used the `type()` function to know which class `empty_set` and `empty_dictionary` belong to.

Duplicate Items in a Set

Let's see what will happen if we try to include duplicate items in a set.

```
numbers = {2, 4, 6, 6, 2, 8}
print(numbers)    # {8, 2, 4, 6}
Run Code
```

Here, we can see there are no duplicate items in the set as a set cannot contain duplicates.

Add and Update Set Items in Python

Sets are mutable. However, since they are unordered, indexing has no meaning.

We cannot access or change an element of a set using indexing or slicing. The set data type does not support it.

Add Items to a Set in Python

In Python, we use the [add\(\)](#) method to add an item to a set. For example,

```
numbers = {21, 34, 54, 12}

print('Initial Set:', numbers)

# using add() method
numbers.add(32)

print('Updated Set:', numbers)
Run Code
```

Output


```
Initial Set: {34, 12, 21, 54}
Updated Set: {32, 34, 12, 21, 54}
```

In the above example, we have created a set named `numbers`. Notice the line,

```
numbers.add(32)
```

Here, `add()` adds **32** to our set.

Update Python Set

The [update\(\)](#) method is used to update the set with items other collection types (lists, tuples, sets, etc). For example,

```
companies = {'Lacoste', 'Ralph Lauren'}
tech_companies = ['apple', 'google', 'apple']

# using update() method
companies.update(tech_companies)

print(companies)

# Output: {'google', 'apple', 'Lacoste', 'Ralph Lauren'}
Run Code
```

Here, all the unique elements of `tech_companies` are added to the `companies` set.

Remove an Element from a Set

We use the [discard\(\)](#) method to remove the specified element from a set. For example,

```
languages = {'Swift', 'Java', 'Python'}

print('Initial Set:', languages)
```

```
# remove 'Java' from a set
removedValue = languages.discard('Java')
```

```
print('Set after remove():', languages)
```

[Run Code](#)

Output

```
Initial Set: {'Python', 'Swift', 'Java'}
Set after remove(): {'Python', 'Swift'}
```

Python Dictionary

A Python dictionary is a collection of items, similar to lists and tuples. However, unlike lists and tuples, each item in a dictionary is a **key-value** pair (consisting of a key and a value).

Create a Dictionary

We create a dictionary by placing `key: value` pairs inside curly brackets `{}`, separated by commas. For example,

```
# creating a dictionary
country_capitals = {
    "Germany": "Berlin",
    "Canada": "Ottawa",
    "England": "London"
}
```

```
# printing the dictionary
```

```
print(country_capitals)
```

[Run Code](#)

Output

```
{'Germany': 'Berlin', 'Canada': 'Ottawa', 'England': 'London'}
```

The `country_capitals` dictionary has three elements (key-value pairs), where `'Germany'` is the key and `'Berlin'` is the value assigned to it and so on.

```
country_capitals = {  
    'Germany': 'Berlin',  
    'Canada': 'Ottawa',  
    'England': 'London'  
}
```



Python Dictionary

Notes:

- Dictionary keys must be immutable, such as tuples, strings, integers, etc. We cannot use mutable (changeable) objects such as lists as keys.
- We can also create a dictionary using a Python built-in function `dict()`. To learn more, visit [Python dict\(\)](#).

Valid and Invalid Dictionaries

Keys of a dictionary must be immutable

Keys of a dictionary must be unique

Access Dictionary Items

We can access the value of a dictionary item by placing the key inside square brackets.

```
country_capitals = {  
    "Germany": "Berlin",  
    "Canada": "Ottawa",  
    "England": "London"  
}  
  
# access the value of keys  
print(country_capitals["Germany"])    # Output: Berlin  
print(country_capitals["England"])    # Output: London  
Run Code
```

Note: We can also use the [get\(\)](#) method to access dictionary items.

Add Items to a Dictionary

We can add an item to a dictionary by assigning a value to a new key. For example,

```
country_capitals = {  
    "Germany": "Berlin",  
    "Canada": "Ottawa",  
}  
  
# add an item with "Italy" as key and "Rome" as its value  
country_capitals["Italy"] = "Rome"  
  
print(country_capitals)
```

[Run Code](#)

Output

```
{'Germany': 'Berlin', 'Canada': 'Ottawa', 'Italy': 'Rome'}
```

Remove Dictionary Items

We can use the [del](#) statement to remove an element from a dictionary. For example,

```
country_capitals = {  
    "Germany": "Berlin",  
    "Canada": "Ottawa",  
}
```

```
# delete item having "Germany" key
del country_capitals["Germany"]

print(country_capitals)
Run Code
```

Output

```
{'Canada': 'Ottawa'}
```

Note: We can also use the [pop\(\)](#) method to remove an item from a dictionary.

If we need to remove all items from a dictionary at once, we can use the [clear\(\)](#) method.

```
country_capitals = {
    "Germany": "Berlin",
    "Canada": "Ottawa",
}

# clear the dictionary
country_capitals.clear()

print(country_capitals)
Run Code
```

Output

```
{}
```

Change Dictionary Items

Python dictionaries are mutable (changeable). We can change the value of a dictionary element by referring to its key. For example,

```
country_capitals = {
    "Germany": "Berlin",
    "Italy": "Naples",
```

```
"England": "London"
}

# change the value of "Italy" key to "Rome"
country_capitals["Italy"] = "Rome"

print(country_capitals)
Run Code
```

Output

```
{'Germany': 'Berlin', 'Italy': 'Rome', 'England': 'London'}
```

Note: We can also use the [update\(\)](#) method to add or change dictionary items.

Iterate Through a Dictionary

A dictionary is an ordered collection of items (starting from Python 3.7), therefore it maintains the order of its items.

We can iterate through dictionary keys one by one using a [for loop](#).

```
country_capitals = {
    "United States": "Washington D.C.",
    "Italy": "Rome"
}

# print dictionary keys one by one
for country in country_capitals:
    print(country)

print()

# print dictionary values one by one
for country in country_capitals:
    capital = country_capitals[country]
    print(capital)
Run Code
```

Output

```
United States
Italy

Washington D.C.
Rome
```

Find Dictionary Length

We can find the length of a dictionary by using the [len\(\)](#) function.

```
country_capitals = {"England": "London", "Italy": "Rome"}
```

```
# get dictionary's length
print(len(country_capitals))    # Output: 2
```

```
numbers = {10: "ten", 20: "twenty", 30: "thirty"}
```

```
# get dictionary's length
print(len(numbers))            # Output: 3
```

```
countries = {}
```

```
# get dictionary's length
print(len(countries))          # Output: 0
```

[Run Code](#)

Python Dictionary Methods

Here are some of the commonly used [dictionary methods](#).

Function	Description
pop()	Removes the item with the specified key.
update()	Adds or changes dictionary items.

clear()	Remove all the items from the dictionary.
keys()	Returns all the dictionary's keys.
values()	Returns all the dictionary's values.
get()	Returns the value of the specified key.
popitem()	Returns the last inserted key and value as a tuple.
copy()	Returns a copy of the dictionary.

Dictionary Membership Test

We can check whether a key exists in a dictionary by using the `in` and `not in` operators.

```
file_types = {
    ".txt": "Text File",
    ".pdf": "PDF Document",
    ".jpg": "JPEG Image",
}

# use of in and not in operators
print(".pdf" in file_types)      # Output: True
print(".mp3" in file_types)     # Output: False
print(".mp3" not in file_types) # Output: True
Run Code
```

Note: The `in` operator checks whether a key exists; it doesn't check whether a value exists or not.

Python Functions

A function is a block of code that performs a specific task.

Suppose we need to create a program to make a circle and color it. We can create two functions to solve this problem:

1. function to create a circle
2. function to color the shape

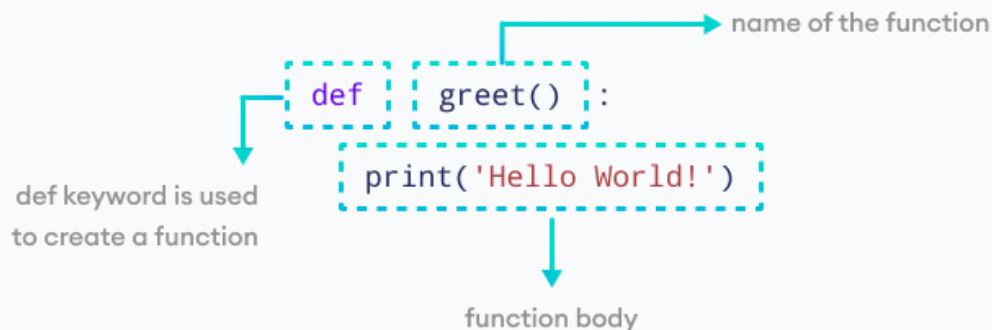
Dividing a complex problem into smaller chunks makes our program easy to understand and reuse.

Create a Function

Let's create our first function.

```
def greet():  
    print('Hello World!')
```

Here are the different parts of the program:



Create a Python Function

Here, we have created a simple function named `greet()` that prints **Hello World!**

Note: When writing a function, pay attention to indentation, which are the spaces at the start of a code line.

In the above code, the `print()` statement is indented to show it's part of the function body, distinguishing the function's definition from its body.

Calling a Function

In the above example, we have declared a function named `greet()`.

```
def greet():  
    print('Hello World!')
```

If we run the above code, we won't get an output.

It's because creating a function doesn't mean we are executing the code inside it. It means the code is there for us to use if we want to.

To use this function, we need to call the function.

Function Call

```
greet()
```

Example: Python Function Call

```
def greet():  
    print('Hello World!')
```

call the function

```
greet()
```

```
print('Outside function')
```

[Run Code](#)

Output

```
Hello World!  
Outside function
```

In the above example, we have created a function named `greet()`. Here's how the control of the program flows:



Working of Python Function

Here,

1. When the function `greet()` is called, the program's control transfers to the function definition.
2. All the code inside the function is executed.
3. The control of the program jumps to the next statement after the function call.

Python Function Arguments

Arguments are inputs given to the function.

```
def greet(name):  
    print("Hello", name)
```

pass argument

```
greet("John")
```

[Run Code](#)

Sample Output 1

```
Hello John
```

Here, we passed 'John' as an argument to the `greet()` function.

We can pass different arguments in each call, making the function re-usable and dynamic.

Let's call the function with a different argument.

```
greet("David")
```

Sample Output 2

Hello David

Example: Function to Add Two Numbers

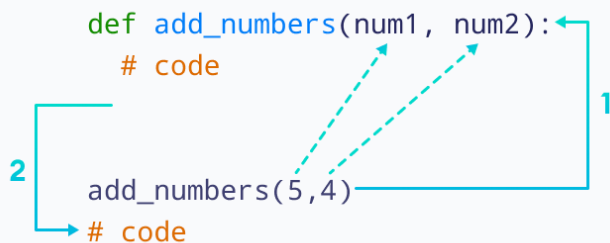
```
# function with two arguments
def add_numbers(num1, num2):
    sum = num1 + num2
    print("Sum: ", sum)

# function call with two values
add_numbers(5, 4)
Run Code
```

Output

Sum: 9

In the above example, we have created a function named `add_numbers()` with arguments: `num1` and `num2`.



Python Function with Arguments

Parameters and Arguments

The return Statement

We return a value from the function using the `return` statement.

```
# function definition
def find_square(num):
```

```
result = num * num
return result

# function call
square = find_square(3)

print('Square:', square)
Run Code
```

Output

Square: 9

In the above example, we have created a function named `find_square()`. The function accepts a number and returns the square of the number.

```
def find_square(num):
    # code
    return result

square = find_square(3)
# code
```

Working

of functions in Python

Note: The `return` statement also denotes that the function has ended. Any code after `return` is not executed.

The pass Statement

The `pass` statement serves as a placeholder for future code, preventing errors from empty code blocks.

It's typically used where code is planned but has yet to be written.

```
def future_function():
    pass
```

```
# this will execute without any action or error
future_function()
Run Code
```

Note: To learn more, visit [Python Pass Statement](#).

Python Library Functions

Python provides some built-in functions that can be directly used in our program.

We don't need to create the function, we just need to call them.

Some Python library functions are:

1. [print\(\)](#) - prints the string inside the quotation marks
2. [sqrt\(\)](#) - returns the square root of a number
3. [pow\(\)](#) - returns the power of a number

These library functions are defined inside the module. And to use them, we must include the module inside our program.

For example, `sqrt()` is defined inside the [math](#) module.

Note: To learn more about library functions, please visit [Python Library Functions](#).

Example: Python Library Function

```
import math

# sqrt computes the square root
square_root = math.sqrt(4)
```

```
print("Square Root of 4 is",square_root)
```

```
# pow() computes the power
```

```
power = pow(2, 3)
```

```
print("2 to the power 3 is",power)
```

[Run Code](#)

Output

```
Square Root of 4 is 2.0
```

```
2 to the power 3 is 8
```

Python Function Arguments

In computer programming, an argument is a value that is accepted by a function.

Before we learn about function arguments, make sure to know about [Python Functions](#).

Example 1: Python Function Arguments

```
def add_numbers(a, b):
```

```
    sum = a + b
```

```
    print('Sum:', sum)
```

```
add_numbers(2, 3)
```

```
# Output: Sum: 5
```

[Run Code](#)

In the above example, the function `add_numbers()` takes two parameters: `a` and `b`. Notice the line,

```
add_numbers(2, 3)
```

Here, `add_numbers(2, 3)` specifies that parameters `a` and `b` will get values **2** and **3** respectively.

Function Argument with Default Values

In Python, we can provide default values to function arguments.

We use the `=` operator to provide default values. For example,

```
def add_numbers( a = 7, b = 8):  
    sum = a + b  
    print('Sum:', sum)  
  
# function call with two arguments  
add_numbers(2, 3)  
  
# function call with one argument  
add_numbers(a = 2)  
  
# function call with no arguments  
add_numbers()  
Run Code
```

Output

```
Sum: 5  
Sum: 10  
Sum: 15
```

In the above example, notice the function definition

```
def add_numbers(a = 7, b = 8):  
    ...
```

Here, we have provided default values **7** and **8** for parameters `a` and `b` respectively. Here's how this program works

1. add_number(2, 3)

Both values are passed during the function call. Hence, these values are used instead of the default values.

2. add_number(2)

Only one value is passed during the function call. So, according to the positional argument **2** is assigned to argument `a`, and the default value is used for parameter `b`.

3. add_number()

No value is passed during the function call. Hence, default value is used for both parameters `a` and `b`.

Python Keyword Argument

In keyword arguments, arguments are assigned based on the name of the arguments. For example,

```
def display_info(first_name, last_name):  
    print('First Name:', first_name)  
    print('Last Name:', last_name)  
  
display_info(last_name = 'Cartman', first_name = 'Eric')  
Run Code
```

Output

```
First Name: Eric  
Last Name: Cartman
```

Here, notice the function call,

```
display_info(last_name = 'Cartman', first_name = 'Eric')
```

Here, we have assigned names to arguments during the function call.

Hence, `first_name` in the function call is assigned to `first_name` in the function definition. Similarly, `last_name` in the function call is assigned to `last_name` in the function definition.

In such scenarios, the position of arguments doesn't matter.

Python Function With Arbitrary Arguments

Sometimes, we do not know in advance the number of arguments that will be passed into a function. To handle this kind of situation, we can use [arbitrary arguments in Python](#).

Arbitrary arguments allow us to pass a varying number of values during a function call.

We use an asterisk (*) before the parameter name to denote this kind of argument. For example,

```
# program to find sum of multiple numbers

def find_sum(*numbers):
    result = 0

    for num in numbers:
        result = result + num

    print("Sum = ", result)

# function call with 3 arguments
find_sum(1, 2, 3)

# function call with 2 arguments
find_sum(4, 9)
Run Code
```

Output

```
Sum = 6  
Sum = 13
```

In the above example, we have created the function `find_sum()` that accepts arbitrary arguments. Notice the lines,

```
find_sum(1, 2, 3)  
  
find_sum(4, 9)
```

Python Variable Scope

In Python, we can declare variables in three different scopes: local scope, global, and nonlocal scope.

A variable scope specifies the region where we can access a [variable](#). For example,

```
def add_numbers():  
    sum = 5 + 4
```

Here, the `sum` variable is created inside the [function](#), so it can only be accessed within it (local scope). This type of variable is called a local variable. Based on the scope, we can classify Python variables into three types:

1. Local Variables
2. Global Variables
3. Nonlocal Variables

Python Local Variables

When we declare variables inside a function, these variables will have a local scope (within the function). We cannot access them outside the function.

These types of variables are called local variables. For example,

```
def greet():  
  
    # local variable  
    message = 'Hello'  
  
    print('Local', message)  
  
greet()  
  
# try to access message variable  
# outside greet() function  
print(message)  
Run Code
```

Output

```
Local Hello  
NameError: name 'message' is not defined
```

Here, the `message` variable is local to the `greet()` function, so it can only be accessed within the function.

That's why we get an error when we try to access it outside the `greet()` function.

To fix this issue, we can make the variable named `message` global.

Python Global Variables

In Python, a variable declared outside of the function or in global scope is known as a global variable. This means that a global variable can be accessed inside or outside of the function.

Let's see an example of how a global variable is created in Python.

```
# declare global variable  
message = 'Hello'  
  
def greet():
```

```
# declare local variable
print('Local', message)
```

```
greet()
print('Global', message)
```

[Run Code](#)

Output

```
Local Hello
Global Hello
```

This time we can access the `message` variable from outside of the `greet()` function. This is because we have created the `message` variable as the global variable.

```
# declare global variable
message = 'Hello'
```

Now, `message` will be accessible from any scope (region) of the program.

Python Nonlocal Variables

In Python, nonlocal variables are used in nested functions whose local scope is not defined. This means that the variable can be neither in the local nor the global scope.

We use the `nonlocal` [keyword](#) to create nonlocal variables. For example,

```
# outside function
def outer():
    message = 'local'

    # nested function
    def inner():

        # declare nonlocal variable
        nonlocal message

        message = 'nonlocal'
        print("inner:", message)
```

```
inner()
print("outer:", message)
```

```
outer()
Run Code
```

Output

```
inner: nonlocal
outer: nonlocal
```

In the above example, there is a nested `inner()` function. We have used the `nonlocal` keywords to create a nonlocal variable.

The `inner()` function is defined in the scope of another function `outer()`

Python Strings

In Python, a string is a sequence of characters. For example, `"hello"` is a string containing a sequence of characters `'h'`, `'e'`, `'l'`, `'l'`, and `'o'`.

We use single quotes or double quotes to represent a string in Python. For example,

```
# create a string using double quotes
string1 = "Python programming"

# create a string using single quotes
string1 = 'Python programming'
```

Here, we have created a string [variable](#) named `string1`. The variable is initialized with the string `"Python Programming"`.

Example: Python String

```
# create string type variables

name = "Python"
```

```
print(name)

message = "I love Python."
print(message)
Run Code
```

Output

```
Python
I love Python.
```

In the above example, we have created string-type variables: `name` and `message` with values `"Python"` and `"I love Python"` respectively.

Here, we have used double quotes to represent strings, but we can use single quotes too.

Access String Characters in Python

We can access the characters in a string in three ways.

- **Indexing:** One way is to treat strings as a [list](#) and use index values. For example,

```
greet = 'hello'

# access 1st index element
print(greet[1]) # "e"
Run Code
```

- **Negative Indexing:** Similar to a list, Python allows negative indexing for its strings. For example,

```
greet = 'hello'

# access 4th last element
print(greet[-4]) # "e"
Run Code
```

- **Slicing:** Access a range of characters in a string by using the slicing operator colon `:`. For example,

```
greet = 'Hello'

# access character from 1st index to 3rd index
print(greet[1:4]) # "ell"
Run Code
```

Note: If we try to access an index out of the range or use numbers other than an integer, we will get errors.

Python Strings are Immutable

In Python, strings are immutable. That means the characters of a string cannot be changed. For example,

```
message = 'Hola Amigos'
message[0] = 'H'
print(message)
Run Code
```

Output

```
TypeError: 'str' object does not support item assignment
```

However, we can assign the variable name to a new string. For example,

```
message = 'Hola Amigos'

# assign new string to message variable
message = 'Hello Friends'

print(message); # prints "Hello Friends"
Run Code
```

Python Multiline String

We can also create a multiline string in Python. For this, we use triple double quotes `"""` or triple single quotes `'''`. For example,

```
# multiline string
message = """
Never gonna give you up
Never gonna let you down
"""
```

```
print(message)
```

[Run Code](#)

Output

```
Never gonna give you up
Never gonna let you down
```

In the above example, anything inside the enclosing triple quotes is one multiline string.

Python String Operations

Many operations can be performed with strings, which makes it one of the most used [data types](#) in Python.

1. Compare Two Strings

We use the `==` operator to compare two strings. If two strings are equal, the operator returns `True`. Otherwise, it returns `False`. For example,

```
str1 = "Hello, world!"
str2 = "I love Swift."
str3 = "Hello, world!"
```

```
# compare str1 and str2
```

```
print(str1 == str2)
```

```
# compare str1 and str3
```

```
print(str1 == str3)
```

[Run Code](#)

Output

```
False
True
```

In the above example,

1. `str1` and `str2` are not equal. Hence, the result is `False`.
2. `str1` and `str3` are equal. Hence, the result is `True`.

2. Join Two or More Strings

In Python, we can join (concatenate) two or more strings using the `+` operator.

```
greet = "Hello, "  
name = "Jack"
```

```
# using + operator  
result = greet + name  
print(result)
```

```
# Output: Hello, Jack  
Run Code
```

In the above example, we have used the `+` operator to join two strings: `greet` and `name`.

Iterate Through a Python String

We can iterate through a string using a [for loop](#). For example,

```
greet = 'Hello'  
  
# iterating through greet string  
for letter in greet:  
    print(letter)  
Run Code
```

Output

```
H  
e  
l
```

```
1  
o
```

Python String Length

In Python, we use the [len\(\)](#) method to find the length of a string. For example,

```
greet = 'Hello'
```

```
# count length of greet string  
print(len(greet))
```

```
# Output: 5
```

[Run Code](#)

String Membership Test

We can test if a substring exists within a string or not, using the keyword `in`.

```
print('a' in 'program') # True  
print('at' not in 'battle') # False
```

Python String `isnumeric()`

The `isnumeric()` method checks if all the characters in the [string](#) are numeric.

Example

```
pin = "523"
```

```
# checks if every character of pin is numeric  
print(pin.isnumeric())
```

```
# Output: True
```

Example 1: Python `isnumeric()`

```
symbol_number = "012345"
```

```
# returns True as symbol_number has all numeric characters
print(string1.isnumeric())

text = "Python3"

# returns False as every character of text is not numeric
print(string2.isnumeric())
```

Output

```
True
False
```

Example 2: isnumeric() with Other Numeric Types

Python treats mathematical characters like numbers, subscripts, superscripts, and characters having Unicode numeric value properties (like a fraction, roman numerals, currency numerators) as numeric characters.

The `isnumeric()` method returns `True` with these characters. For example:

```
# string with superscript
superscript_string = '²3455'
print(superscript_string.isnumeric())

# string with fraction value
fraction_string = '½123'
print(fraction_string.isnumeric())
```

[Run Code](#)

Output

```
True
```

True

Python String find()

The `find()` method returns the index of first occurrence of the substring (if found). If not found, it returns **-1**.

Example

```
message = 'Python is a fun programming language'

# check the index of 'fun'
print(message.find('fun'))

# Output: 12
```

Python String index()

The `index()` method returns the index of a substring inside the [string](#) (if found). If the substring is not found, it raises an [exception](#).

Example

```
text = 'Python is fun'

# find the index of is
result = text.index('is')

print(result)

# Output: 7
```

Python String isdecimal()

The syntax of `isdecimal()` is

```
string.isdecimal()
```

isdecimal() Parameters

The `isdecimal()` doesn't take any parameters.

Return Value from isdecimal()

The `isdecimal()` returns:

- **True** if all characters in the [string](#) are decimal characters.
- **False** if at least one character is not decimal character.

Example 1: Working of isdecimal()

```
s = "28212"
print(s.isdecimal())

# contains alphabets
s = "32ladk3"
print(s.isdecimal())

# contains alphabets and spaces
s = "Mo3 nicaG e1 l22er"
print(s.isdecimal())
Run Code
```

Output

```
True
False
False
```

The superscript and subscripts are considered digit characters but not decimals. If the string contains these characters (usually written using unicode), `isdecimal()` returns `False`.

Similarly, roman numerals, currency numerators and fractions are considered numeric numbers (usually written using unicode) but not decimals.

The `isdecimal()` also returns `False` in this case.

There are two methods `isdigit()` and `isnumeric()` that checks whether the string contains digit characters and numeric characters respectively.

Learn more about [isdigit\(\)](#) and [isnumeric\(\)](#) methods.

Example 2: String Containing digits and Numeric Characters

```
s = '23455'
print(s.isdecimal())

#s = '²3455'
s = '\u00B23455'
print(s.isdecimal())

# s = '½'
s = '\u00BD'
print(s.isdecimal())
```

[Run Code](#)

Output

```
True
False
False
```

Python String `isprintable()`

The `isprintable()` method returns `True` if all characters in the [string](#) are printable. If not, it returns `False`.

Example

```
text = 'apple'
```

```
# returns True if text is printable
result = text.isprintable()

print(result)

# Output: True
```

Python Classes and Objects

In the last tutorial, we learned about [Python OOP](#). We know that Python also supports the concept of objects and classes.

An object is simply a collection of data ([variables](#)) and methods ([functions](#)). Similarly, a class is a blueprint for that object.

Before we learn about objects, let's first learn about classes in Python.

Python Classes

A class is considered a blueprint of objects.

We can think of the class as a sketch (prototype) of a house. It contains all the details about the floors, doors, windows, etc.

Based on these descriptions, we build the house; the house is the object.

Since many houses can be made from the same description, we can create many objects from a class.

Define Python Class

We use the `class` [keyword](#) to create a class in Python. For example,

```
class ClassName:
    # class definition
```

Here, we have created a class named `ClassName`.

Let's see an example,

```
class Bike:
    name = ""
    gear = 0
```

Here,

1. `Bike` - the name of the class
2. `name/gear` - variables inside the class with default values `""` and `0` respectively.

Note: The variables inside a class are called attributes.

Python Objects

An object is called an instance of a class.

Suppose `Bike` is a class then we can create objects like `bike1`, `bike2`, etc from the class.

Here's the syntax to create an object.

```
objectName = ClassName()
```

Let's see an example,

```
# create class
class Bike:
    name = ""
    gear = 0
```

```
# create objects of class
bike1 = Bike()
```

Here, `bike1` is the object of the class. Now, we can use this object to access the class attributes.

Access Class Attributes Using Objects

We use the `.` notation to access the attributes of a class. For example,

```
# modify the name property
bike1.name = "Mountain Bike"

# access the gear property
bike1.gear
```

Here, we have used `bike1.name` and `bike1.gear` to change and access the value of `name` and `gear` attributes, respectively.

Example 1: Python Class and Objects

```
# define a class
class Bike:
    name = ""
    gear = 0

# create object of class
bike1 = Bike()

# access attributes and assign new values
bike1.gear = 11
bike1.name = "Mountain Bike"

print(f"Name: {bike1.name}, Gears: {bike1.gear} ")
Run Code
```

Output

```
Name: Mountain Bike, Gears: 11
```

In the above example, we have defined the class named `Bike` with two attributes: `name` and `gear`.

We have also created an object `bike1` of the class `Bike`.

Finally, we have accessed and modified the properties of an object using the `.` notation.

Create Multiple Objects of Python Class

We can also create multiple objects from a single class. For example,

```
# define a class
class Employee:
    # define a property
    employee_id = 0

# create two objects of the Employee class
employee1 = Employee()
employee2 = Employee()

# access property using employee1
employee1.employeeID = 1001
print(f"Employee ID: {employee1.employeeID}")

# access properties using employee2
employee2.employeeID = 1002
print(f"Employee ID: {employee2.employeeID}")
```

[Run Code](#)

Output

```
Employee ID: 1001
Employee ID: 1002
```

In the above example, we have created two objects `employee1` and `employee2` of the `Employee` class.

Python Methods

We can also define a function inside a Python class. A Python function defined inside a class is called a **method**.

Let's see an example,

```
# create a class
class Room:
    length = 0.0
    breadth = 0.0

    # method to calculate area
    def calculate_area(self):
        print("Area of Room =", self.length * self.breadth)

# create object of Room class
study_room = Room()

# assign values to all the properties
study_room.length = 42.5
study_room.breadth = 30.8

# access method inside class
study_room.calculate_area()
Run Code
```

Output

```
Area of Room = 1309.0
```

In the above example, we have created a class named `Room` with:

1. **Attributes:** `length` and `breadth`
2. **Method:** `calculate_area()`

Here, we have created an object named `study_room` from the `Room` class.

We then used the object to assign values to attributes: `length` and `breadth`.

Notice that we have also used the object to call the method inside the class,

```
study_room.calculate_area()
```

Here, we have used the `.` notation to call the method. Finally, the statement inside the method is executed.

Python Constructors

Earlier we assigned a default value to a class attribute,

```
class Bike:
    name = ""
    ...
# create object
bike1 = Bike()
```

However, we can also initialize values using the constructors. For example,

```
class Bike:

    # constructor function
    def __init__(self, name = ""):
        self.name = name

bike1 = Bike()
```

Here, `__init__()` is the constructor function that is called whenever a new object of that class is instantiated.

The constructor above initializes the value of the `name` attribute.

We have used the `self.name` to refer to the `name` attribute of the `bike1` object.

If we use a constructor to initialize values inside a class, we need to pass the corresponding value during the object creation of the class.

```
bike1 = Bike("Mountain Bike")
```

Here, `"Mountain Bike"` is passed to the `name` parameter of `__init__()`.

Also Read:

- [Python inheritance](#)

- [Polymorphism in Python](#)
- [Python object\(\)](#)
- [Python classmethod\(\)](#)

Python Inheritance

Being an object-oriented language, Python supports class inheritance. It allows us to create a new class from an existing one.

- The newly created class is known as the **subclass** (child or derived class).
- The existing class from which the child class inherits is known as the superclass (parent or base class).

Python Inheritance Syntax

```
# define a superclass
class super_class:
    # attributes and method definition

# inheritance
class sub_class(super_class):
    # attributes and method of super_class
    # attributes and method of sub_class
```

Here, we are inheriting the `sub_class` from the `super_class`.

Note: Before you move forward with inheritance, make sure you know how [Python classes and objects](#) work.

Example: Python Inheritance

```
class Animal:
```

```
# attribute and method of the parent class
name = ""

def eat(self):
    print("I can eat")

# inherit from Animal
class Dog(Animal):

    # new method in subclass
    def display(self):
        # access name attribute of superclass using self
        print("My name is ", self.name)

# create an object of the subclass
labrador = Dog()

# access superclass attribute and method
labrador.name = "Rohu"
labrador.eat()

# call subclass method
labrador.display()
Run Code
```

Output

```
I can eat
My name is  Rohu
```

In the above example, we have derived a subclass `Dog` from a superclass `Animal`. Notice the statements,

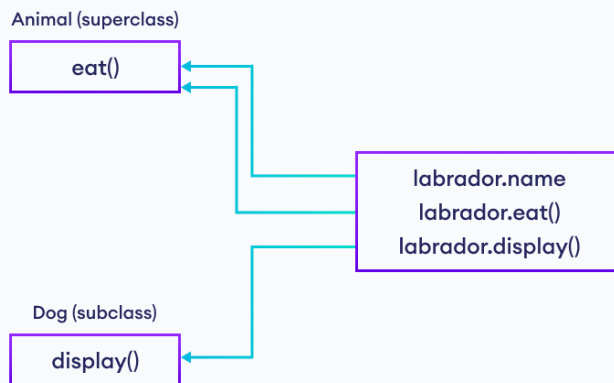
```
labrador.name = "Rohu"

labrador.eat()
```

Here, we are using `labrador` (object of `Dog`) to access `name` and `eat()` of the `Animal` class.

This is possible because the subclass inherits all attributes and methods of the superclass.

Also, we have accessed the `name` attribute inside the method of the `Dog` class using `self`.



Python Inheritance Implementation

is-a relationship

Inheritance is an **is-a** relationship. That is, we use inheritance only if there exists an **is-a** relationship between two classes. For example,

- **Car** is a **Vehicle**
- **Apple** is a **Fruit**
- **Cat** is an **Animal**

Here, **Car** can inherit from **Vehicle**, **Apple** can inherit from **Fruit**, and so on.

Method Overriding in Python Inheritance

In the previous example, we see the object of the subclass can access the method of the superclass.

However, what if the same method is present in both the superclass and subclass?

In this case, the method in the subclass overrides the method in the superclass. This concept is known as method overriding in Python.

Example: Method Overriding

```
class Animal:

    # attributes and method of the parent class
    name = ""

    def eat(self):
        print("I can eat")

# inherit from Animal
class Dog(Animal):

    # override eat() method
    def eat(self):
        print("I like to eat bones")

# create an object of the subclass
labrador = Dog()

# call the eat() method on the labrador object
labrador.eat()
Run Code
```

Output

```
I like to eat bones
```

In the above example, the same method `eat()` is present in both the `Dog` class and the `Animal` class.

Now, when we call the `eat()` method using the object of the `Dog` subclass, the method of the `Dog` class is called.

This is because the `eat()` method of the `Dog` subclass overrides the same method of the `Animal` superclass.

The `super()` Function in Inheritance

Previously we saw that the same method ([function](#)) in the subclass overrides the method in the superclass.

However, if we need to access the superclass method from the subclass, we use the `super()` function. For example,

```
class Animal:

    name = ""

    def eat(self):
        print("I can eat")

# inherit from Animal
class Dog(Animal):

    # override eat() method
    def eat(self):

        # call the eat() method of the superclass using super()
        super().eat()

        print("I like to eat bones")

# create an object of the subclass
labrador = Dog()

labrador.eat()
Run Code
```

Output

```
I can eat
I like to eat bones
```

In the above example, the `eat()` method of the `Dog` subclass overrides the same method of the `Animal` superclass.

Inside the `Dog` class, we have used

```
# call method of superclass
super().eat()
```

to call the `eat()` method of the `Animal` superclass from the `Dog` subclass. So, when we call the `eat()` method using the `labrador` object

```
# call the eat() method
labrador.eat()
```

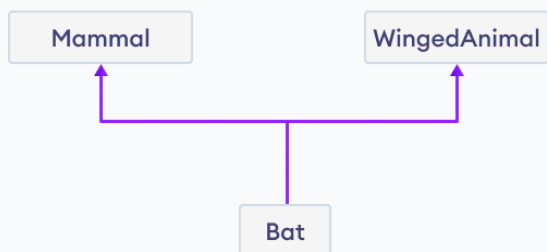
Both the overridden and the superclass version of the `eat()` method is executed.

To learn more, visit [Python super\(\)](#).

Python Multiple Inheritance

A [class](#) can be derived from more than one superclass in Python. This is called multiple [inheritance](#).

For example, a class `Bat` is derived from superclasses `Mammal` and `WingedAnimal`. It makes sense because bat is a mammal as well as a winged animal.



Multiple Inheritance

Python Multiple Inheritance Syntax

```
class SuperClass1:
    # features of SuperClass1

class SuperClass2:
```

```
# features of SuperClass2

class MultiDerived(SuperClass1, SuperClass2):
    # features of SuperClass1 + SuperClass2 + MultiDerived class
```

Here, the `MultiDerived` class is derived from `SuperClass1` and `SuperClass2` classes.

Example: Python Multiple Inheritance

```
class Mammal:
    def mammal_info(self):
        print("Mammals can give direct birth.")

class WingedAnimal:
    def winged_animal_info(self):
        print("Winged animals can flap.")

class Bat(Mammal, WingedAnimal):
    pass

# create an object of Bat class
b1 = Bat()

b1.mammal_info()
b1.winged_animal_info()
Run Code
```

Output

```
Mammals can give direct birth.
Winged animals can flap.
```

In the above example, the `Bat` class is derived from two super classes: `Mammal` and `WingedAnimal`. Notice the statements,

```
b1 = Bat()
b1.mammal_info()
b1.winged_animal_info()
```

Here, we are using `b1` (object of `Bat`) to access `mammal_info()` and `winged_animal_info()` methods of the `Mammal` and the `WingedAnimal` class respectively.

Python Multilevel Inheritance

In Python, not only can we derive a class from the superclass but you can also derive a class from the derived class. This form of inheritance is known as **multilevel inheritance**.

Here's the syntax of the multilevel inheritance,

```
class SuperClass:
    # Super class code here

class DerivedClass1(SuperClass):
    # Derived class 1 code here
    # Derived class 2 code here
```

Here, the `DerivedClass1` class is derived from the `SuperClass` class, and the `DerivedClass2` class is derived from the `DerivedClass1` class.



Multilevel Inheritance in Python

Example: Python Multilevel Inheritance

```
class SuperClass:

    def super_method(self):
        print("Super Class method called")

# define class that derive from SuperClass
class DerivedClass1(SuperClass):
    def derived1_method(self):
        print("Derived class 1 method called")

# define class that derive from DerivedClass1
class DerivedClass2(DerivedClass1):

    def derived2_method(self):
        print("Derived class 2 method called")

# create an object of DerivedClass2
d2 = DerivedClass2()

d2.super_method() # Output: "Super Class method called"
```

```
d2.derived1_method() # Output: "Derived class 1 method called"

d2.derived2_method() # Output: "Derived class 2 method called"
Run Code
```

Output

```
Super Class method called
Derived class 1 method called
Derived class 2 method called
```

In the above example, `DerivedClass2` is derived from `DerivedClass1`, which is derived from `SuperClass`.

It means that `DerivedClass2` inherits all the attributes and methods of both `DerivedClass1` and `SuperClass`.

Hence, we are using `d2` (object of `DerivedClass2`) to call methods from `SuperClass`, `DerivedClass1`, and `DerivedClass2`.

Method Resolution Order (MRO) in Python

If two superclasses have the same method ([function](#)) name and the derived class calls that method, Python uses the MRO to search for the right method to call. For example,

```
class SuperClass1:
    def info(self):
        print("Super Class 1 method called")

class SuperClass2:
    def info(self):
        print("Super Class 2 method called")

class Derived(SuperClass1, SuperClass2):
    pass

d1 = Derived()
d1.info()

# Output: "Super Class 1 method called"
```

[Run Code](#)

Here, `SuperClass1` and `SuperClass2` both of these classes define a method `info()`.

So when `info()` is called using the `d1` object of the `Derived` class, Python uses the **MRO** to determine which method to call.

In this case, the **MRO** specifies that methods should be inherited from the leftmost superclass first, so `info()` of `SuperClass1` is called rather than that of `SuperClass2`.